A Comparative Analysis between AI Generated Code and Human Written Code: A Preliminary Study

Abhi Patel
Department of Computer Science
Rutgers University
New Brunswick, NJ, USA
ap2101@scarletmail.rutgers.edu

Kazi Zakia Sultana
School of Computing
Montclair State University
Montclair, NJ, USA
sultanak@montclair.edu

Bharath K. Samanthula
School of Computing
Montclair State University
Montclair, NJ, USA
samanthulab@montclair.edu

2024 IEEE International Conference on Big Data (BigData) | 979-8-3503-6248-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/BigData62323.2024.10825958

***Abstract*—In today's world where generative artificial intelligence has almost become an integral part of the coding, new challenges must be faced. Therefore, evaluating software bugs in both human written and AI generated code can be useful for the developers. A comparative analysis of these two coding practices is not only helpful for the decisions taken by the developers, it will also assist to give a direction on how to improve the quality of AI driven code. Currently, researchers have leveraged the role of software metrics to compare human written code and AI generated code as these metrics have long been utilized for software bug and vulnerability prediction by the researchers. They also analyzed the secure coding practices in terms of the number of bugs found in both AI and human written code. Our study is an extension of the current works as this study focuses on a set of metrics and a set of bugs as identified by some static analyzer tools. Investigating these two coding practices from different angles can help to reveal unknown relationships and factors that further can be analyzed to improve code quality of recent AI tools. Therefore, the main objective of our work is to identify the relationships between software metrics and bugs in AI generated code and human written code to compare and contrast the coding profiles of the two approaches. This will offer developers critical knowledge to enhance their strategies in mitigating potential bug risks across different coding methodologies. We have utilized top-rated Java solutions to 90 LeetCode problems, generated corresponding AI Java solutions to them, and utilized various static analysis tools to collect metrics and bugs. In this study, we found that two software metrics $CountLineCodeDecl$ and $CountLineCodeExe$ are positively correlated with the bug $DLS\_DEAD\_LOCAL\_STORE$ and the metric $AvgCyclomatic$ is related to the bug $AvoidLiteralsInIfCondition$ in both human written and AI generated code. These findings provide developers with critical insights into potential bug risks, enabling more effective mitigation strategies across different coding methodologies.**

***Index Terms*—Artificial Intelligence, Static Code Analysis, Bugs, Software Metrics**

I. Introduction

As Artificial Intelligence (AI) becomes an increasingly prevalent part of our lives, so does its use as a tool to assist software developers in generating code. While platforms like StackOverflow have existed for years, with the advent of AI tools such as Github Copilot and ChatGPT, developers have been utilizing these tools for scripting to automate processes such as testing and thereby ensure efficient software development practices [1]. Software developers are benefited by the AI tools in different tasks. However, researchers need to address different concerns and challenges related to the use of AI tools in software development. According to Vaidya et al. [2], flawed training data can result in vulnerabilities such as buffer overflows in AI-generated code. In particular, similar to human-written code, developers must constantly be aware of inefficient and bug-ridden code generated by AI to ensure the viability of their overall products. A study [3] was able to find 10 specific bug patterns common in three Large Language Models (LLMs) as verified by LLM experts. This study gives developers an insight into what bugs to look out for while using AI generated code.

Software metrics used to quantify different attributes of software have been prevalent in the use of vulnerability prediction models [4]–[11]. This means developers should keep track of them, as any variation can hint at changes in the security of code [12]–[14]. An example of an important software metric is Cyclomatic Complexity which measures the number of linearly independent paths in a piece of code [15]. Ebert [15] continues to state that Cyclomatic Complexity correlates directly with the number of test cases required to cover the code, and thus a lower maintainability and higher defect density as well. Thus, it is very important to study variations in software metrics in order to find correlations with specific bugs in code, as this will further help the developers to identify and fix vulnerable code.

In case of AI generated code, the relationships between software metrics and bugs may not be same as the relationships found in human written code. Therefore, this paper aims to look at the relationship between software metrics and bugs in human written code as well as in the AI generated counterparts. In our work, we have used various static code analysis tools to gather a list of software metrics and bugs, and then found patterns between the two for both AI and human written code. *The goal of this study is to better understand the*

National Science Foundation under the REU Site grant CNS-2050548

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.
979-8-3503-6248-0/24/$31.00 ©2024 IEEE

relationship between software metrics and bugs and how they might differ in AI and human-written code. The findings of our study will bolster the comparative analysis of AI generated and human written code which will create a new door of research on the challenges of the generative AI in software development.

The **major contributions** of this paper are as follows:

- We have systematically compared instances of bugs in code generated by (whether human or AI) and their counterparts in order to establish that certain bug-metric relationships are consistent in both human and AI code. This relationship can further be used for predicting bugs in AI code.
- Developers will gain the ability to make informed decisions about potential issues arising from specific bug-metric relationships, taking into account whether the code was generated by AI or written by humans.
- We have also compared the relationships between the software metrics and bug density in both AI generated and and human-written code. This finding will assist developers in making their decision of using AI tools for software development in different contexts.

II. Related Work

In the section, we will discuss existing works related to software metrics, vulnerabilities, and bugs common in AI code.

Tihanyi et al. [16] introduced the FormAI dataset of AI-generated C code that are classified by vulnerabilities. They used GPT-3.5-turbo in order to generate the code, and utilized a unique prompting technique based on a set of different types and styles available to complete a prompt template. The FormAI dataset introduced in their work can be used to help in vulnerability prediction. Asare et al. [17] conducted a study to find whether Github Copilot is capable of producing vulnerable code and how that code can be compared to the human written vulnerable code. They designed their experiments by collecting vulnerable C/C++ codes from the Common Vulnerabilities and Exposures (CVE) datasets and then prompting GitHub Copilot with the same code minus the buggy lines in order to see if it would introduce similar vulnerabilities. They found that Github Copilot generates similar vulnerabilities in code. Our study is inspired by the earlier research on analyzing AI generated code in terms of vulnerability [16], [17]. In our study, we have focused on Java solutions generated by AI tool and compared them with human written code in terms of code characteristics and bugs standpoint.

Liu et al. [18] conducted a study to analyze the correctness, complexity, and security aspects of the ChatGPT generated codes. They investigated the bug fixing abilities of ChatGPT and how this process can affect correctness, complexity, and security aspects of codes. They also looked at the impact on the time of release of coding problems. They utilized LeetCode coding problems as well as Common Weakness Enumeration (CWE) database to collect vulnerabilities. One of their key results is that ChatGPT is better at generating correct code for problems released before 2021 than after 2021 (as ChatGPT knowledge base is built up to 2021). Another key result was that although ChatGPT could gradually correct buggy code, Cyclomatic Complexity also increased with the rounds of fixing through ChatGPT. In another study [19], Liu et al. analyzed the ChatGPT generated code in java and python and investigated code correctness in terms of task difficulty and programming language. They also checked for the quality as well as the maintainability of the code. They concluded that AI code is full of compilation/runtime errors and maintainability issues, as well as task difficulty and program size definitely affect the quality of generated code. In order to conduct a competitive study, we have employed cyclomatic complexity metric as one of our software metrics that can characterize the code and aimed to build a correlation with the likeliness of software bugs. In our future study, we plan to consider the correctness of code before comparing them with human written code.

Tosi et al. [20] used three specific coding problems from a university course that were prompted to GPT-3, GPT-4, and Bard. SonarQube and SonarCloud were used to collect vulnerabilities and metrics for the generated codes. They concluded that the three LLMs provided similar solutions without any bugs. This implies that LLMs follow similar standards in generating codes and need appropriate human prompts to generate right codes. Hamer et al. [21] compared code generated by ChatGPT with that of the corresponding StackOverflow answers. Their goal was to compare the usefulness of these two platforms to the developers. They found that ChatGPT resulted in 20% fewer vulnerabilities than StackOverflow. In general, they concluded that developers need to be more cautious in the code taken from either platform [21].

Idrisov and Schlippe [22] compared maintainability metrics such as Cyclomatic Complexity and Lines of Code as well as Halstead Complexity of AI generated code and human code written in three different programming languages. They used the latest problems from LeetCode and compared the metrics across multiple different LLMs. They created their own prompting technique in order to correct code. Thus, they took into account whether or not the code generated was correct, and then calculated a maintainability index to measure the fix ability of incorrect code. Overall, the researchers concluded that AI is still very far from creating code that is efficient, correct, and maintainable at all times. However, they also concluded that only small modifications are needed to turn incorrect code into correct one as generated by AI tools. As their prompting technique seemed to be comprehensive in generating the best possible codes from LLMs, our study utilized their prompting technique in order to generate AI code for LeetCode problems.

In this paper, we conducted a preliminary study on the comparative analysis of AI generated code vs. human written code. Our work aims to do this analysis with respect to code characteristics as defined by some software metrics. We believe results of our study on AI generated code and human

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

written code from another perspective can help developers to take important decisions in their development as well as this study will further advance research in this area.

III. Methodology

Figure 1 presents steps followed in our experimental methodology.

A. Research Goal and Questions

The objective of this research is to identify relationships between software metrics and bugs that exist in both AI generated code and human written code. We have collected a dataset of human written code to a set of problems, and utilized AI to generate code for the same problems. Then, we extracted software metrics as well as bugs using various static code analysis tools. To reach the goal, we have formulated three research questions and tried to answer these questions based on our experimental findings.

Research Question 1 (RQ1): How can we relate software metrics to the bugs that exist in AI generated code and human written code? RQ1 allows us to identify potential patterns of relationships between software metrics and bugs in two types of coding practices: AI generated vs. human written. Earlier studies [4], [6], [10] have evaluated the effectiveness of the software metrics for vulnerability prediction. Although researchers frequently use metrics-based prediction models, many false-positives are introduced as the metrics are not directly related to coding constructs. In our study, we have focused on identifying relationships between metrics and bugs in both AI generated and human written codes so that we can compare these two types of code in terms of these relationships and thereby can detect how metrics can eventually be used for bugs prediction in AI generated tools.

Research Question 2 (RQ2): Is there any significant difference between AI generated and human written codes in terms of code size and complexity? To answer this question, we have analyzed the values of two metrics: $AvgCountLineCode$ (Average number of lines of code) and $AvgCyclomatic$ (Average Cyclomatic Complexity) in both code counterparts. The reason to analyze these metrics is to understand the code characteristics of both types of codes in terms of their size and complexity. It will give us an idea to compare these two types of coding practices so that it can be acted as a deciding factor for the future developers.

Research Question 3 (RQ3): Is there any significant difference between AI generated and human written codes in terms of bug density? This question will address the significant differences between AI generated code and its human code counterpart in terms of the number of bugs they contain. The importance of this question lies in understanding how one coding practice (either using AI tool or coding manually by human) can overwhelm another in terms of correctness and if it can make any impact on the programmer's coding practices.

B. Program Code Collection

In the study by Idrisov et al. [22], the researchers utilized 18 problems with 6 problems of each difficulty level (easy, medium, hard) from LeetCode. They ensured that the selected problems were recent, so they would not appear in the training datasets of any of the generative AIs being tested. This was done to lower the likelihood that the generative AIs would produce answers based on previously encountered data, thereby providing a more accurate assessment of the performance of each of the generative AIs considered. When they created the corresponding AI solutions for the LeetCode problems for each of the generative AIs in their experiments, they came up with their own prompting technique. This prompting technique consists of four possible steps as stated by Idrisov et al. [22]:

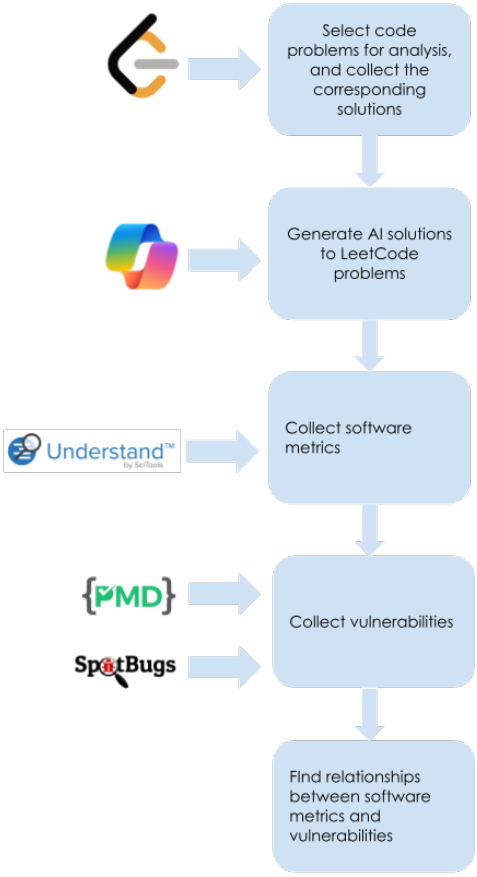

Fig. 1. Methodology

> "1. We prompted 'Solve the following problem with code.' together with the original LeetCode coding problem description, examples, example explanations, and constraints.
> 2. If the result with prompt 1 was incorrect program code (i.e., did not solve the problem) or was not executable, we removed the example explanations.
> 3. If the result with prompt 2 was incorrect program code (i.e., did not solve the problem) or was not executable, we removed the examples.
> 4. If the result with prompt 3 was incorrect program code (i.e., did not solve the problem) or was not executable, we took the prompt from prompts 1–3

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

that resulted in program code that was closest to our human-generated reference" [22].

In our data collection, we utilized many steps similar to those followed by Idrisov et al. Since we wanted to limit potential confounding variables such as the differences in patterns in different programming languages as well as varying performances of different generative AIs, we modified their data collection process to focus on only one programming language and one generative AI. We have collected 90 LeetCode problems in total including 30 latest problems from each of the three categories of difficulty levels: easy, medium, and hard. We have chosen the most voted solutions by the LeetCode users considering them as the most liked or popular solutions by the coders. In our study, we have selected Microsoft Copilot as the generative AI as it utilizes GPT-4. We also focused on the Java language considering its compatibility with many static code analysis tools. Since we are only using one generative AI, we have used 90 total problems so that we have a good pool of data to discover any patterns of relations. We have also tested the correctness of the AI-generated solutions by running the solutions directly on LeetCode and determining if it passes all test cases or not. Our goal is to collect potentially enlightening statistics about the correctness (whether or not the program fulfills its intended function) of different AI-generated solutions. We have utilized the prompting strategy used in the study by Idrisov et al. to gather the dataset of corresponding AI code for the various LeetCode problems.

C. Software Metrics and Bugs Collection

To obtain the software metrics, we first needed to save the code as files. We used JDoodle, an online compiler, as an intermediary tool to parse and download the code quickly [23]. Since the Java code was sourced from LeetCode, most of the class names were simply "Solution". To differentiate each file based on the corresponding problem and whether the solution was generated by a human or AI, we had to rename the class names to match the filenames, which we formatted as code type, problem number, difficulty level (ex. ai3148medium). This step ensured that the code was executable. Finally, we saved the modified code as files.

To extract software metrics, we have used Understand tool by Sci Tools [24]. We opened all the files containing the codes (AI and human) as a folder in Understand. Then we have exported all the metrics to a CSV file, including for all classes as well as files, although we only considered file level metrics. Some of the software metrics that we collected are listed in Table I. The software metrics relevant to our results are listed in Table II. Figure 2 showcases the collection of the metrics using the Understand tool.

To find the patterns of relationships between software metrics and bugs, we considered the software metrics that can be more interesting and can answer our respective research question effectively. $AvgCyclomatic$ and $AvgCountLineCode$, as shown in Table II have been selected for this analysis. These two metrics are the most relevant indicators of code complexity as they provide information related to the number of decision statements and the number of lines respectively. The selection of AvgCyclomatic and AvgCountLineCode can be justified as these metrics effectively capture key aspects of code complexity. AvgCountLineCode indicates the size of the code, which can impact maintainability, while AvgCyclomatic measures the complexity of decision-making paths, correlating with potential bug occurrences. Analyzing these metrics together provides a comprehensive understanding of how code complexity relates to software defects. We also considered two more metrics $CountLineCodeDecl$ and $CountLineCodeExe$ as presented in Table II. According to our consensus, we finally selected these four metrics for answering our first two research questions.

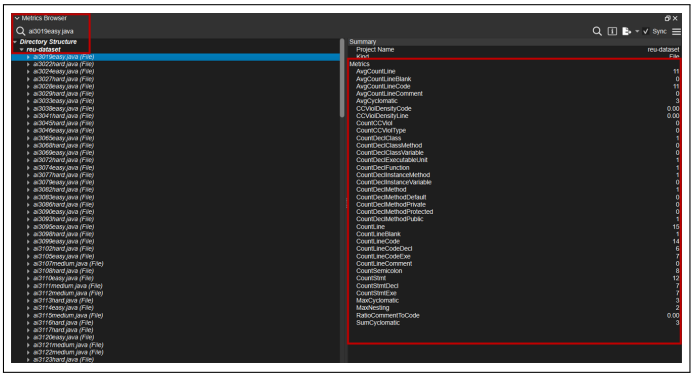

Fig. 2. Understand Metrics Browser Window

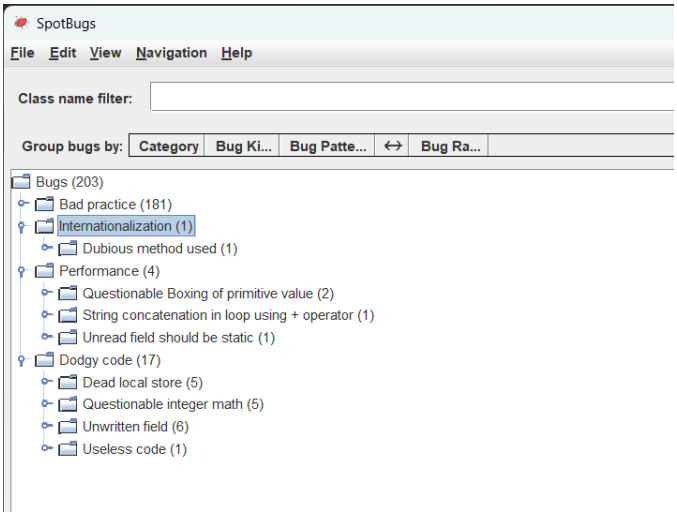

Fig. 3. SpotBugs bugs window

We have utilized SpotBugs [25] as well as PMD [26] to collect bugs from the sets of codes. SpotBugs is a static code analyzer that scans byte code, and hence we had to execute all the Java files in order to get byte code for the files [25]. Figure 3 presents the bugs window of the tool SpotBugs. PMD is another static code analyzer that scans the source code for identifying and locating bugs [26]. We ran PMD utilizing the command line and used the proper commands to store the found bugs as a csv file. An important note is that we had

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

to exclude the $LoosePackageCoupling$ rule as it requires explicit configuration, unlike other rules.

Our rationale for using SpotBugs and PMD for identifying bugs is that they are complementary static code analyzers targeting different types of issues. SpotBugs excels at analyzing compiled bytecode to detect potential errors, such as null pointer dereferences and memory leaks, that could lead to runtime failures. PMD, on the other hand, analyzes source code to enforce stylistic and structural standards. However, these tools have limitations: they cannot detect dynamic bugs that only appear during runtime, such as race conditions or deadlocks, and may miss complex logic errors, certain security vulnerabilities, and performance issues. Nonetheless, this combination provides a solid foundation for bug detection and can be expanded in the future by incorporating more diverse analysis tools.

After completing the extraction of bugs and metrics, we had CSV files consisting of metrics, bugs found by SpotBugs, and bugs found by PMD. Some of the bugs that we collected for this study are listed in Table III. The bugs relevant to our results are listed in Table IV.

TABLE I
SOFTWARE METRICS

Metrics	Description
CountDeclClassMethod	Number of class methods
CountDeclMethodDefault	Number of local methods
CountDeclClass	Number of classes
MaxNesting	Maximum nesting level of control constructs
CountDeclInstanceVariable	Number of instance variables [aka NIV]
CountDeclClassVariable	Number of class variables
CountLineCode	Number of lines containing source code [aka LOC]
MaxCyclomaticStrict	Maximum strict cyclomatic complexity of nested functions or methods

TABLE II
SOFTWARE METRICS USED IN THIS STUDY

Metrics	Description
AvgCountLineCode	Average number of lines containing source code for all nested functions or methods
AvgCyclomatic	Average cyclomatic complexity for all nested functions or methods
CountLineCodeDecl	Number of lines containing declarative source code
CountLineCodeExe	Number of lines containing executable source code

D. Data Analysis

For both PMD and SpotBugs, we have not considered any bugs that were detected for violating the rule of class naming conventions as we specifically altered them in order to make the code executable. Specifically, for SpotBugs, we ended up removing any instances of $NM\_CLASS\_NAMING\_CONVENTION$. For PMD, we removed any instances of $ClassNamingConventions$, but we also removed instances (only 2) of $UnnecessaryImport$. The reason we decided to exclude $UnnecessaryImport$ from the reported bugs is that both AI-generated and human-written code display inconsistencies in how imports can be handled (whether declared individually or with an asterisk). This inconsistency does not significantly impact the functionality of the code and can be considered a stylistic choice rather than a critical bug. Besides that, we have reported instances of $ShortClassName$ because they indicate potential issues with code readability and maintainability, and there was no impact by us on the length of the class names where this bug was reported.

To address RQ1, we set out to directly compare the metrics of code with a specific bug against its bug-free counterpart. For instance, if a particular bug appeared in human-written code but not in the respective AI-generated code, we have considered them side by side allowing us to analyze patterns of bug-metric relationship and track differences. In cases where human and AI-generated code exhibited the same bug, we would still compare them to observe whether they followed consistent patterns across occurrences. We determine that there is a pattern of relationship between a specific metric and a specific bug for both human-written and AI-generated code, if there is a change between the metrics for the inflicted code and the non-inflicted code and if that trend stays consistent for most instances. We kept track of the number of code pairs in which this pattern holds true and used that to determine if a particular pattern is significant. We narrowed our bug search to some specific bugs based on their significant relation with the complexity of the code. For example, $AvoidLiteralsInIfCondition$ considers if statements which could have a significant impact on the complexity of the code. Some of the bugs that we collected are presented in Table III as well as the bugs relevant to our results are shown in Table IV.

To address RQ2, we compared the AI-generated code and human-written code side by side for each of the 90 problems and determined which code had the higher $AvgCountLineCode$ and $AvgCyclomatic$.

To address RQ3, we defined our own metric $BugDensity$ by dividing the total number of bugs by $AvgCountLineCode$ (Average number of lines of code) metric in a file. To answer our third research question, we measured the $BugDensity$ metric for all AI-generated codes and their human counterparts in case of 90 problems and determined which code had the higher bug density.

IV. RESULTS AND DISCUSSION

A. RQ1: How can we relate software metrics to the bugs that exist in AI generated code and human written code?

For RQ1, we found two significant results that highlight a relationship between software metrics and bugs that seems to be consistent in both human-written and AI-generated code.

One of them is between the security bug $DLS\_DEAD\_LOCAL\_STORE$ and the metrics $CountLineCodeExe$ and $CountLineCodeDecl$. We analyzed four code pairs. In one pair, both of AI and human written codes contain the bug on the other hand, in remaining three pairs, only one counterpart (AI generated or human written) contains the bug. We found elevated values for the two metrics $CountLineCodeExe$ and $CountLineCodeDecl$ in the buggy counterpart of these three pairs. We also noticed that the single pair where both counterparts exhibit the bug, these two metrics present almost similar values in both of them. For example, as shown in Figure 4, AI generated and human written buggy codes have the values of 5 and 4 respectively for CountLineCodeDecl metric. In Figure 5, AI

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EXAMPLES OF BUGS

Bug (Tool)	Description
AvoidReassigningParameters (PMD)	Occurs when you reassign values to parameters passed in.
LocalVariableCouldBeFinal (PMD)	Occurs when a local variable (inside a method or block) is assigned a value once and isn't reassigned afterward, indicating that it could safely be marked as `final`.
LooseCoupling (PMD)	Occurs when you use excessive coupling to implementation type for collections.
MethodArgumentCouldBeFinal (PMD)	Occurs when a method or constructor parameter isn't reassigned within the method body, indicating it could safely be marked as `final`.
BX_UNBOXING_IMMEDIATELY_REBOXED (SpotBugs)	Occurs when a boxed value is unboxed and immediately reboxed.
IM_BAD_CHECK_FOR_ODD (SpotBugs)	Occurs when the code uses `x % 2 == 1` to check for odd, which doesn't work for negative numbers.
SBSC_USE_STRINGBUFFER_CONCATENATION (SpotBugs)	Occurs specifically when string concatenation is performed repeatedly in a loop using the + operator.
UC_USELESS_CONDITION (SpotBugs)	Occurs when the condition has no effect as it produces the same value that the relevant variable already was.

TABLE IV
BUGS USED IN THIS STUDY

Bug (Tool)	Description
AvoidLiteralsInIfCondition (PMD)	Occurs when you use hard-coded literals in conditionals.
DLS_DEAD_LOCAL_STORE (SpotBugs)	Occurs when a value is assigned to a local variable, but is never used or read.

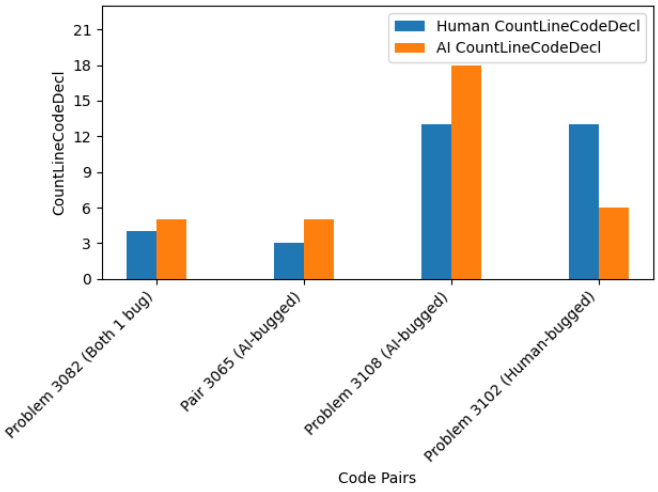

Fig. 4. $CountLineCodeDecl$ and $DLS\_DEAD\_LOCAL\_STORE$

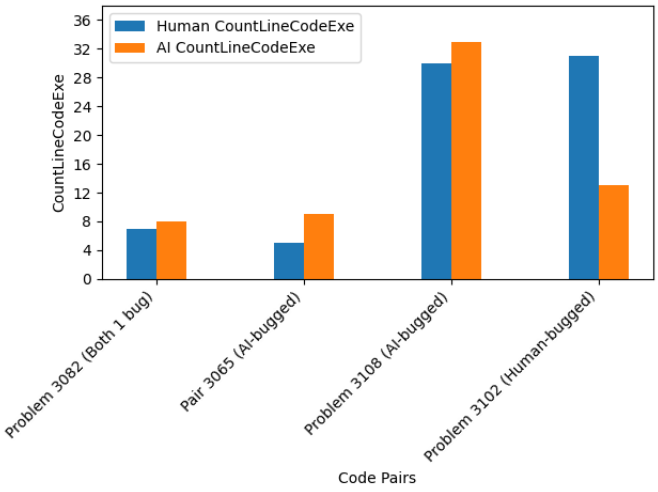

Fig. 5. $CountLineCodeExe$ and $DLS\_DEAD\_LOCAL\_STORE$

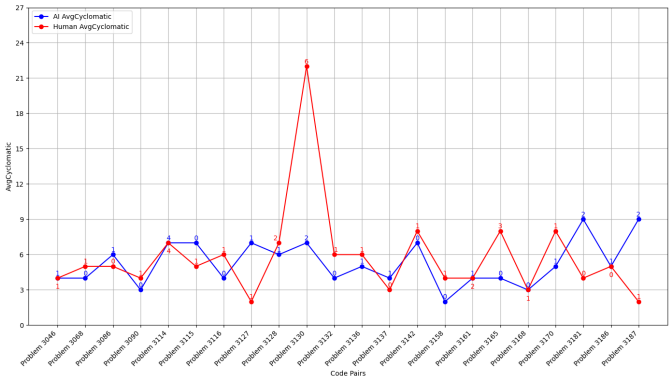

Fig. 6. $AvgCyclomatic$ and $AvoidLiteralsInIfCondition$

generated and human written buggy codes have the values of 8 and 7 respectively for the CountLineCodeExe metric. This showcase determines a very strong pattern of relationship between the bug $DLS\_DEAD\_LOCAL\_STORE$ and two aforementioned metrics. The presence of this bug elevates the value for these two metrics irrespective of the coding practices (AI or human written). On the other hand, both coding practices behave same with respect to these two characteristics when both of them contain that bug.

$DLS\_DEAD\_LOCAL\_STORE$ occurs when a value is assigned to a local variable, but never used before that variable is reassigned to something else. Our findings on positive correlation between this security bug and two metrics $CountLineCodeExe$ (number of executable code lines) and $CountLineCodeDecl$ (number of declarative code lines) can easily be interpreted. With respect to $CountLineCodeExe$, it makes sense that the higher the number of executable code lines are, there is a greater likelihood of a variable being reassigned in order to coordinate with executable code. With respect to $CountLineCodeDecl$, since declaration of variables is a part of declarative code, it is natural that if variables are reassigned, there would be a higher value for the metric $CountLineCodeDecl$.

We identified another relationship between the bug $AvoidLiteralsInIfCondtion$ and $AvgCyclomatic$ metric.

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

Out of 17 pairs in which one counterpart (either AI generated or human written) has more $AvoidLiteralsInIfCondition$ bugs, 13 pairs showcase a higher value for $AvgCyclomatic$ (e.g. higher the number of bugs, the higher the respective value for $AvgCyclomatic$). Out of the remaining four pairs that don't meet this pattern, three pairs have the same value for $AvgCyclomatic$ metric and one pair in which AI-generated code has higher $AvgCyclomatic$ value than human-written code. In addition to the 17 pairs, there were five pairs where each counterpart had the same amount of bugs. Among these, two pairs had the same $AvgCyclomatic$ values for both AI generated and human written code, two pairs resulted in the human written code having a higher value for $AvgCyclomatic$ than the AI generated code, and the final pair resulted in the AI generated code having a higher value for $AvgCyclomatic$ than the human written code. Figure 6 presents the distribution of $AvgCyclomatic$ metric over the 22 pairs of codes where the bug $AvoidLiteralsInIfCondition$ was found.

The $AvoidLiteralsInIfCondition$ bug occurs whenever there are hard-coded literals in conditionals. As cyclomatic complexity represents the number of linearly independent paths through code, it has ties to the amount of conditionals in a piece of code. When the number of conditionals in a program gets higher, there is a high chance that hardcoded literals may also show up in the conditionals.

B. RQ2: Is there any significant difference between AI generated and human written codes in terms of code size and complexity?

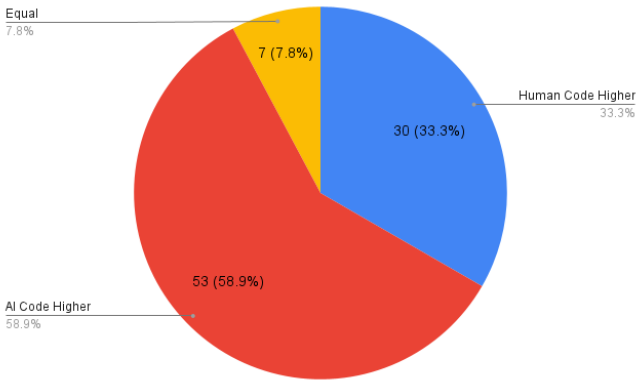

Fig. 7. AvgCountLineCode distribution

To answer this question, we analyzed the distribution of two metrics $AvgCountLineCode$ (Average number of lines of code) and $AvgCyclomatic$ (Average Cyclomatic Complexity) in two coding practices. We found 30 code pairs in our dataset where the human written code has higher $AvgCountLineCode$ value than the AI generated counterpart. On the other hand, in case of 53 instances, AI generated code has higher $AvgCountLineCode$ value than the human written counterpart, and in seven instances, AI generated and human written counterparts exhibit the same value for

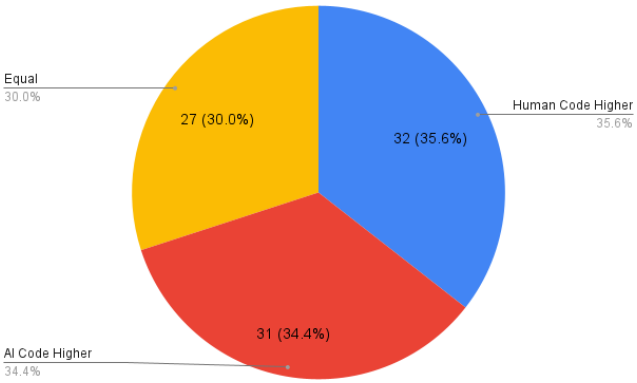

Fig. 8. AvgCyclomatic Distribution

$AvgCountLineCode$ metric. For $AvgCyclomatic$ (Average Cyclomatic Complexity) metric, we figured that human written code shows higher value in 32 instances while AI generated code shows higher value in 31 instances. The distribution of $AvgCountLineCode$ and $AvgCyclomatic$ have been presented in Figures 7 and 8 respectively.

For $AvgCountLineCode$, our findings reveal that, in around 60% cases, AI generated code presents higher value than human written code. Although there can be multiple factors that will better describe this finding, based on human cognitive behavior, we can interpret that humans may prefer conciseness of code while AI tools may focus on other factors while solving a problem. Further research can analyze this finding to understand other relevant factors. For $AvgCyclomatic$, it seems that the distribution is roughly even between AI generated code and human written code. It means that we cannot definitely determine the pattern of $AvgCyclomatic$ metric in case of AI generated and human written codes. We diagnosed that the average difference of this metric between AI and human code (both in cases where the metric is higher in AI generated code than human written code and vice versa) is around 2 which indicates that generative AI and human coding practices have almost same complexity level. Further research can delve into this finding as well to discover any underlying causes.

C. RQ3: Is there any significant difference between AI generated and human written codes in terms of bug density?

$BugDensity$ is computed by dividing the total number of bugs by $AvgCountLineCode$ (Average number of lines of code) metric in a code. We found 24 code pairs where AI generated code has higher $BugDensity$ values than the human written counterpart, 62 pairs where human code has higher $BugDensity$ values than the AI generated code, and four pairs where both of them has the same value for $BugDensity$ metric. One significant thing to note here is that it seems $BugDensity$ is overall lower for AI generated code than human written code. To dive deeper into this, we identified 42 AI generated code instances which are correct and 48

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

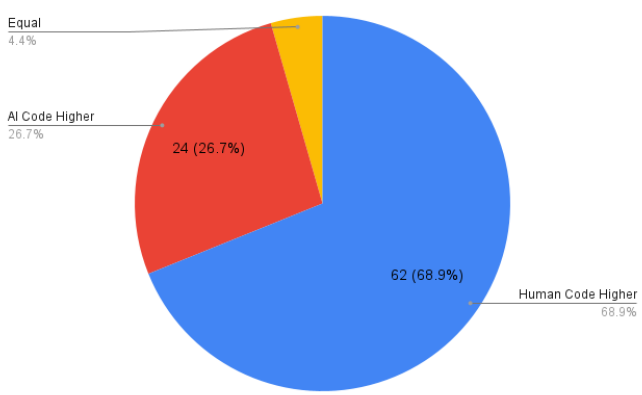

Fig. 9. Bug Density distribution

instances which are incorrect. More specifically, out of the 42 which are correct, the difficulty levels are as follows: easy: 25, medium: 12, and hard: 5. Out of the 48 problems that are not correct, the difficulty levels are as follows: easy: 5, medium: 18, and hard: 25. Since 52% of the incorrect solutions are categorized as hard problems and only around 11.9% of the correct solutions are categorized as hard problems, it seems that the AI tends to generate more correct solutions for the easier problems from LeetCode than for the harder ones. On the other hand, AI generates more incorrect solutions for the harder problems. It should be noted that since the human written codes were collected from LeetCode based on popular choices, the solutions can be considered as correct and functional.

Our finding that AI generated code results in fewer bug density than human generated code can follow the results presented by Hamer et al. [21] where they found that ChatGPT generated code resulted in fewer vulnerabilities than code retrieved from StackOverflow. Based on our findings, it can be implied that there can be a correlation between code correctness and number of bugs in AI generated code since AI generated code has an overall lower number of bugs compared to human written code. We aim to investigate this finding further in our future study.

V. Threats to Validity

A. Internal Validity

This threat suggests possibilities of unexpected relationships among variables. One confounding variable can include the variance of expertise of the user who submitted the top-voted solution in LeetCode. This could have an impact on the number of bugs that shows up in code, and this factor has not been explored in our study. There are various other confounding factors that we have not considered in the study. Another confounding factor can include the correctness of code. For this study, we collected correctness of code for each sample, however we didn't use it in the experiment.

B. Construct Validity

This threat relates to the precision with which an experiment measures what it claims to measure. One potential concern here could be with respect to the selection of metrics and bugs. This study did not consider all possible software metrics and bugs reported by the static analyzer tools, and thus may not represent more effective relationships between metrics and bugs. As a preliminary study, based on our mutual agreement, we selected a set of metrics and bugs which could have potential impact on AI and human generated code. In addition, some potential patterns may have been ignored due to the limited scope of human analysis.

C. External Validity

This threat relates to the ability to generalize the results of the study. The results cannot necessarily be generalized to broader codebases, as we mainly utilized LeetCode problems for this study. Since solutions to LeetCode problems can differ from professional codebases, the findings may not be generalized in broader contexts. Also, as our study was limited to using only one generative AI tool and therefore, our results may not be effectively extended to other generative AIs which are also popular and widely used. However, this study provides a great starting point for further research in this area.

VI. Conclusion and Future Plans

In this study, we compiled software metrics as well as bugs for coding solutions of 90 selected LeetCode problems. We selected the top-voted Java based solutions (human written) to those LeetCode problems and generated AI solutions for each of those selected problems. Once we compiled the metrics and bugs for the human written and AI generated codes, we looked for any potential relationships between the chosen software metrics and identified bugs in the codes. We investigated to discover these bug-metric relationships in order to find how they can differ AI coding from human coding practice.

We found that there are two major software metric-bugs relationships that are common to both human written and AI generated codes. According to our study findings, the metrics $CountLineCodeExe$ and $CountLineCodeDecl$ are positively correlated with the bug $DLS\_DEAD\_LOCAL\_STORE$. Our study also finds that the metric $AvgCyclomatic$ is positively correlated with the $AvoidLiteralsInIfCondition$ bug in both coding practices.

Our study also focused on three metrics $AvgCountLineCode$, $AvgCyclomatic$, and $BugDensity$ for identifying how code size, code complexity and density of bugs vary in both coding practices (AI generated vs. human written). We found that for most LeetCode problems, the $AvgCountLineCode$ metric was higher for AI generated counterpart. Regarding the $AvgCyclomatic$ metric, the ratios are equally distributed in case of AI and human generated codes (34.4% vs. 35.6%). For $BugDensity$, we observed that it was higher for human written code than for AI generated code in most of the LeetCode problems.

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.

In the future, we aim to find more effective relationships between software metrics and bugs by utilizing machine learning and data mining techniques. We also plan to apply some statistical techniques to find the significance of these relationships from statistical point of view. We also aim to extend the search to other languages such as Python and compare how these relationships might differ across different language platforms. We aim to incorporate correctness of code into determining certain relationships (e.g. potential tradeoffs between correctness and security).

VII. Acknowledgments

This research has been supported by the National Science Foundation under the REU Site grant CNS-2050548.

References

[1] C. Ebert and P. Louridas, "Generative ai for software practitioners," *IEEE Software*, vol. 40, no. 4, pp. 30–38, 2023.

[2] J. Vaidya and H. Asif, "A critical look at ai-generate software: Coding with the new ai tools is both irresistible and dangerous," *IEEE Spectrum*, vol. 60, no. 7, pp. 34–39, 2023.

[3] F. Tambon, A. M. Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, and G. Antoniol, "Bugs in large language models generated code: An empirical study," 2024. [Online]. Available: https://arxiv.org/abs/2403.08937

[4] Y. Shin and L. Williams, "An empirical model to predict security vulnerabilities using code complexity metrics," in *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '08. NY, USA: ACM, 2008, pp. 315–317.

[5] Y. Shin, "Exploring complexity metrics as indicators of software vulnerability," in *The 3rd International Doctoral Symposium on Empirical Software Engineering (IDoESE 2008) , co-located with ESEM-2008*, 2008.

[6] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, "Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities," *IEEE Trans. Softw. Eng.*, vol. 37, no. 6, pp. 772–787, nov 2011.

[7] Y. Shin and L. Williams, "Can traditional fault prediction models be used for vulnerability prediction?" *Empirical Software Engineering*, vol. 18, no. 1, pp. 25–59, 2013.

[8] ——, "Is complexity really the enemy of software security?" in *Proceedings of the 4th ACM Workshop on Quality of Protection*, ser. QoP '08. NY, USA: ACM, 2008, pp. 47–50.

[9] I. Chowdhury, B. Chan, and M. Z. Chowdhury, "Security metrics for source code structures," in *Proceedings of the Fourth International Workshop on Software Engineering for Secure Systems*, ser. SESS '08, 2008, pp. 57–64.

[10] I. Chowdhury and M. Zulkernine, "Can complexity, coupling, and cohesion metrics be used as early indicators of vulnerabilities?" in *Proceedings of the 2010 ACM Symposium on Applied Computing*, ser. SAC '10. NY, USA: ACM, 2010, pp. 1963–1969.

[11] ——, "Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities," *J. Syst. Archit.*, vol. 57, no. 3, pp. 294–313, mar 2011.

[12] K. Z. Sultana, V. Anu, and T.-Y. Chong, "Using software metrics for predicting vulnerable classes and methods in java projects: A machine learning approach," *Journal of Software: Evolution and Process*, vol. 33, no. 3, p. e2303, 2021, e2303 smr.2303. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.2303

[13] A. Zhou, K. Z. Sultana, and B. K. Samanthula, "Investigating the changes in software metrics after vulnerability is fixed," in *2021 IEEE International Conference on Big Data (Big Data)*, 2021, pp. 5658–5663.

[14] E. Maza and K. Z. Sultana, "Identifying evolution of software metrics by analyzing vulnerability history in open source projects," in *2022 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*, 2022, pp. 223–232.

[15] C. Ebert, J. Cain, G. Antoniol, S. Counsell, and P. Laplante, "Cyclomatic complexity," *IEEE Software*, vol. 33, no. 6, pp. 27–29, 2016.

[16] N. Tihanyi, T. Bisztray, R. Jain, M. A. Ferrag, L. C. Cordeiro, and V. Mavroeidis, "The formai dataset: Generative ai in software security through the lens of formal verification," in *Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering*, ser. PROMISE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 33–43. [Online]. Available: https://doi.org/10.1145/3617555.3617874

[17] O. Asare, M. Nagappan, and N. Asokan, "Is github's copilot as bad as humans at introducing vulnerabilities in code?" *Empirical Softw. Engg.*, vol. 28, no. 6, sep 2023. [Online]. Available: https://doi.org/10.1007/s10664-023-10380-1

[18] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? assessing the quality of code generation by chatgpt," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, 2024.

[19] Y. Liu, T. Le-Cong, R. Widyasari, C. Tantithamthavorn, L. Li, X.-B. D. Le, and D. Lo, "Refining chatgpt-generated code: Characterizing and mitigating code quality issues," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, jun 2024. [Online]. Available: https://doi.org/10.1145/3643674

[20] D. Tosi, "Studying the quality of source code generated by different ai generative engines: An empirical evaluation," *Future Internet*, vol. 16, no. 6, 2024. [Online]. Available: https://www.mdpi.com/1999-5903/16/6/188

[21] S. Hamer, M. d'Amorim, and L. Williams, "Just another copy and paste? comparing the security vulnerabilities of chatgpt generated code and stackoverflow answers," 2024. [Online]. Available: https://arxiv.org/abs/2403.15600

[22] B. Idrisov and T. Schlippe, "Program code generation with generative ais," *Algorithms*, vol. 17, no. 2, 2024. [Online]. Available: https://www.mdpi.com/1999-4893/17/2/62

[23] JDoodle, "JDoodle - free Online Compiler, Editor for Java, C/C++, etc." *Www.jdoodle.com*, n.d., https://www.jdoodle.com/online-java-compiler/.

[24] "scitools: Visualize your code," https://scitools.com/, Last accessed on 2024-07-29.

[25] "Spotbugs - findbugs' successor. a tool for static analysis to look for bugs in java code," https://github.com/spotbugs/spotbugs, accessed: 2024-07-29.

[26] "Pmd source code analyzer project," https://pmd.github.io/, accessed: 2024-07-29.

Authorized licensed use limited to: TECHNISCHE UNIVERSITAT BRAUNSCHWEIG. Downloaded on June 27,2026 at 16:16:26 UTC from IEEE Xplore. Restrictions apply.